

EJERCICIO Diagrama de clases

01/12/2025

Entornos de Desarrollo

Ejercicio 9 - Tema 3



Vais a diseñar y programar un pequeño simulador simplificado de un **sistema espacial** en el que aparecen:

- Cuerpos celestes naturales (planetas, satélites naturales, cometas...)
- Estructuras creadas por el ser humano (naves espaciales, satélites artificiales, cohetes...)
- Agrupaciones de objetos (sistemas planetarios, constelaciones de satélites...)
- Elementos de organización y control (centros de control de misión)

Todo ello debe representarse **primero con un diagrama UML de clases** y después con **clases en Python**.

Pista general: intenta detectar qué conceptos comparten muchos atributos en común (eso suele dar lugar a una superclase) y qué cosas “contienen” a otras o las “agrupan” (suelen ser composición o agregación).

Parte 1: Modelo de dominio (para UML + Python consola)

Lee con atención la descripción. No está escrito “esta es una clase”, pero casi puedes oírlo. Tienes que extraer:

Qué clases hay.

Qué atributos tiene cada una.

Qué métodos básicos podrían tener.

Qué relaciones existen entre ellas (herencia, composición, agregación, asociación).

1. Objetos naturales del sistema

En el simulador hay varios **cuerpos celestes** de tipo natural. Todos ellos comparten cierta información básica:

Un **nombre** (por ejemplo, “Tierra”, “Luna”, “Halley”...).

El **tipo de cuerpo** (planeta, satélite natural, cometa...).

El **nombre del sistema estelar** al que pertenecen (por ejemplo, “Sistema Solar”).

Una **masa** aproximada.

La **posición actual** en el plano 2D de la simulación (coordenadas x e y en millones de km).

La **velocidad actual** (componentes v_x y v_y).

A partir de estos datos, el simulador puede actualizar su posición con un método del estilo “avanzar un paso de tiempo”.

Dentro de los cuerpos celestes naturales se distinguen varios tipos concretos:

Los que se consideran **planetas** tienen además:

Radio medio.

Número máximo de satélites naturales permitidos en el modelo.

Indicador de si tienen atmósfera densa o no.

Los que actúan como **satélites naturales** (por ejemplo, lunas):

Guardan el nombre del cuerpo al que orbitan.

Mantienen la distancia media a ese cuerpo.

Los que se comportan como **cometas**:

Tienen un período orbital en años.

Un indicador de si en este momento muestran **cola visible** o no.

Pista de herencia: todos estos objetos naturales comparten muchos atributos y comportamiento; piensa en una estructura común de la que “hereden” los demás.

2. Estructuras espaciales creadas por el ser humano

En el simulador también hay **estructuras espaciales artificiales** que se mueven y actúan:

Todas ellas comparten:

Un **identificador** o nombre de misión.

La **agencia responsable** (por ejemplo, “ESA”, “NASA”, “JAXA”...).

El **país principal** que lidera el proyecto.

Una **posición** y una **velocidad** en el mismo sistema de coordenadas que los cuerpos celestes naturales.

Un **estado operativo** (por ejemplo: “En lanzamiento”, “En órbita”, “Fuera de servicio”).

Entre las estructuras espaciales se distinguen varios tipos:

Algunas funcionan como **satélites artificiales**:

Guardan el nombre del cuerpo celeste alrededor del que orbitan.

Tienen una altura de órbita aproximada (en km).

Indican su función principal (comunicación, observación, navegación...).

Otras son **cohetes** que se usan para lanzar cargas al espacio:

Tienen un empuje total.

Indican su capacidad máxima de carga (en kg).

Llevan un contador del número de lanzamientos realizados.

Pista de herencia: vuelve a fijarte en los atributos comunes a todas las estructuras espaciales frente a los que son específicos.

3. Sistemas internos de las estructuras espaciales

Algunas estructuras espaciales (por ejemplo, cohetes y satélites artificiales) cuentan con **sistemas internos** sin los cuales no tendrían sentido. Se pueden considerar parte “interna” de la estructura:

Un **sistema de propulsión** que almacena:

Tipo de combustible.

Cantidad de combustible actual.

Empuje máximo.

Un método que permita “consumir combustible” y otro que indique si queda combustible.

Un **sistema de comunicaciones** que guarda:

Potencia máxima de transmisión.

Frecuencias soportadas.

Indicador de si está operativo o averiado.

Un método para “enviar mensaje” (puede ser un simple `print()`).

Si la estructura espacial que los contiene se pierde o se destruye, estos sistemas internos desaparecen del modelo.

Pista de composición: estos sistemas no tienen sentido fuera de su estructura; piensa en una relación muy fuerte, donde la vida de unos depende de la de otros.

4. Agrupaciones de objetos

En el simulador se trabajan con **agrupaciones** de cuerpos:

Por un lado, hay **sistemas planetarios** que contienen:

Un nombre (por ejemplo, “Sistema Solar”).

Un cuerpo principal (la estrella, que en este caso puedes representar solo por su nombre).

Una colección de cuerpos celestes naturales asociados (planetas, satélites naturales, cometas...) que orbitan en ese sistema.

Por otro lado, se pueden gestionar **constelaciones de satélites artificiales**:

Un nombre de constelación (por ejemplo, “Red de observación polar”).

Un tipo de órbita común.

Una lista de estructuras espaciales que forman parte de la constelación.

Es importante que los cuerpos celestes y las estructuras espaciales puedan **existir aunque no estén dentro de ninguna agrupación**, y que puedan cambiar de una agrupación a otra.

Pista de agregación: las agrupaciones “tienen” o “agrupan” objetos, pero esos objetos no dejan de existir si la agrupación se destruye.

5. Organización y asociación con centros de control

Cada estructura espacial artificial está ligada a un **centro de control de misión**:

Cada centro de control tiene:

Nombre.

País donde se encuentra.

Número de operadores humanos disponibles.

El centro de control puede:

Enviar órdenes a una estructura espacial concreta.

Consultar su estado operativo.

Una misma estructura espacial **solo está controlada por un centro de control a la vez**, pero el centro puede encargarse de varias misiones distintas a lo largo del tiempo.

Si por cualquier motivo una estructura espacial se destruye o deja de funcionar, el centro de control sigue existiendo.

Pista de asociación: aquí hay relación “uno a uno” (en un momento dado) entre misión y centro de control, pero ninguno forma parte “dentro” del otro.

Ejercicio 1 — Diagrama UML de clases

Con toda esta descripción:

Identifica las clases que aparecen en el dominio (tanto naturales como artificiales, sistemas internos, agrupaciones, centro de control...).

Para cada clase, decide:

Sus **atributos**

Sus **métodos**

Dibuja el **diagrama UML de clases** incluyendo:

Herencia

Composición

Agregación

Asociación

Puedes usar draw.io o la herramienta UML que prefieras.

Ejercicio 2 — Programa en Python (consola)

A partir del diagrama UML que has diseñado:

Implementa en Python todas las clases del modelo.

Crea al menos:

Un sistema planetario con varios cuerpos naturales.

Una constelación con varios satélites artificiales.

Un centro de control que gestione algunas estructuras espaciales.

Inicializa todas las posiciones y velocidades con valores sencillos (por ejemplo, enteros).

Imprime por pantalla un **estado inicial**:

Datos de cada cuerpo celeste.

Datos de cada estructura espacial y su centro de control asociado.

Contenido de los sistemas internos (propulsión, comunicaciones...).

Cambia algunas posiciones.

Haz que algún cohete consuma combustible.

Cambia el estado de algún satélite (por ejemplo, de “En lanzamiento” a “En órbita”).

Vuelve a imprimir el **estado final**

Que se vean los cambios en posiciones, estados, combustible, etc.

.

Ejercicio 3 - Simulador visual con Tkinter (y “vibe coding”)

En un segundo ejercicio, debéis reutilizar las mismas clases para crear una **interfaz gráfica** muy sencilla con Tkinter (y, si queréis, pygame u otra librería para sonido):

En este ejercicio si habrá simulación en el tiempo, es decir, las condiciones de los objetos, como su posición, variarán en el tiempo en función de su velocidad y posición anterior.

Cread una ventana principal con:

Un **Canvas** donde se representen:

Algunos cuerpos celestes naturales (por ejemplo, planetas y satélites naturales) como círculos de distinto color.

Algunas estructuras espaciales (satélites artificiales, cohetes) como iconos pequeños.

Un panel lateral con información textual (nombre, tipo, velocidad...).

Añadid botones o controles para:

“Avanzar un paso de tiempo”.

“Iniciar animación” (actualiza posición cada cierto intervalo).

“Mostrar datos de un objeto seleccionado”.

Haced que:

Al avanzar el tiempo, se cambien las coordenadas de los objetos en el Canvas según su velocidad.

Opcionalmente, se reproduzca un sonido cuando un cohete se lanza o cuando un satélite entra en órbita.

La estructura del código debe seguir usando las mismas clases del modelo de dominio; la interfaz solo debe **dibujar** y **pedir datos** a esas clases.

Idea de trabajo “vibe coding” : id probando pequeños cambios en la interfaz mientras veis cómo se mueven los objetos en pantalla; no hace falta que todo esté perfecto desde el minuto uno. Lo importante es que entendáis cómo el modelo (clases) y la vista (Tkinter) se comunican.

Instrucciones: Sube el fichero .xml de draw.io del Ejercicio1 , así como el código del Ejercicio 2 y Ejercicio 3 a tu repositorio de GitHub y pásame el enlace en un documento en Moodle.